PRUEBA DE VULNERABILIDAD / FLUJO DE NEGOCIO

Integridad del carrito OWASP Juice Shop

TC-BL-001 · Manipulación de cantidades · 20 de octubre de 2026

Entorno local oficial 20.3.0 · Java + RestAssured + JUnit + Allure + ZAP

Automatización	Resultado de seguridad	ZAP pasivo
1 prueba aprobada 0 fallos / 0 errores	VULNERABLE quantity = -1 persistida	1 tipo de alerta media 2 tipos informativos

Resultado principal

La API aceptó una cantidad negativa en el carrito del usuario de prueba. Una lectura independiente confirmó la persistencia de **-1**. El valor inicial y la cantidad restablecida fueron **1**. La automatización pasó porque su objetivo es reproducir y documentar la debilidad.

Estado de la entrega

Completados: caso, código ejecutable, ejecución satisfactoria por consola, reporte Allure, reporte ZAP y este documento con evidencias.

Pendientes de acreditación: captura de ejecución satisfactoria dentro de IntelliJ IDEA y publicación/verificación de visibilidad pública en GitHub. Este documento no acredita esos dos requisitos como cumplidos.

Enlace del repositorio de destino

https://github.com/nohpedro/vulneravity_test

Es la URL configurada como origin en el proyecto. La consulta pública inicial devolvió 404; otras conexiones a GitHub agotaron su tiempo. Publicación y acceso público no confirmados.

La demo solicitada respondió HTTP 503; las pruebas documentadas se ejecutaron sobre la imagen oficial local. Todoist quedó excluido por la selección expresa del usuario.

INFORME TÉCNICO / 02

Caso de prueba y criterios

Campo	Definición
ID / objetivo	TC-BL-001. Comprobar que una cantidad negativa no pueda alterar la integridad del carrito.
Alcance	Únicamente el carrito del usuario efímero creado por el caso; sin pedidos, pagos ni modificación de usuarios ajenos.
Precondiciones	Docker activo; Juice Shop 20.3.0 y ZAP 2.17.0 listos; producto 1 disponible; JDK 17+; dependencias Maven.
Datos	Producto 1; cantidad inicial 1; cantidad inválida -1. Usuario qa-UUID@example.invalid y contraseña aleatoria generada en memoria.
Regla de negocio	La cantidad debe ser un entero positivo. Una petición inválida debe rechazarse sin modificar la cantidad válida.
Resultado seguro esperado	HTTP 400 o 422; GET posterior conserva quantity=1.
Oráculo de reproducción	Solo PASS si PUT devuelve 200 y GET posterior devuelve quantity=-1. No basta el estado HTTP.

Paso	Acción	Verificación
1	Registrar usuario e iniciar sesión	Registro 201; login 200; token y carrito propios.
2	POST /api/BasketItems con cantidad 1	HTTP 200; GET /rest/basket/{id} devuelve 1.
3	PUT /api/BasketItems/{id} con -1	Registrar estado HTTP y respuesta.
4	GET independiente del carrito	Confirmar persistencia negativa o rechazo seguro.
5	Restablecer cantidad 1	PUT 200 y GET confirma 1.

El caso se automatiza en un único método JUnit. Sus pasos y evidencias se registran en Allure. ZAP inspecciona el mismo flujo HTTP de forma pasiva; no ejecuta spider ni ataques automáticos.

INFORME TÉCNICO / 03

Evidencia del defecto y ejecución

Dato observado	Resultado
Ejecución final (Bolivia, UTC-4)	20/10/2026, 20:49:54 - 20:49:56
Carrito / ítem	7 / 10 (propios del usuario efímero)
Antes / después / restablecido	1 / -1 / 1
PUT de manipulación	HTTP 200
Precio unitario / cálculo de línea	1.99 / -1.99
Estado final	VULNERABLE; limpieza quantity_restored_to_1

```
PUT /api/BasketItems/10
Content-Type: application/json
Authorization: Bearer [OMITIDO]

{"quantity": -1}

Respuesta: HTTP 200
GET /rest/basket/7
Campo observado: data.Products[0].BasketItem.quantity = -1
```

El importe -1,99 se calcula como precio × cantidad a partir de los datos leídos. No se ejecutó checkout: no se demuestra un cargo, abono ni pérdida monetaria real.

Salida real del ejecutor Maven

```
TC-BL-001: VULNERABLE
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
Time elapsed: 2.448 s -- in security.NegativeQuantityTest
BUILD SUCCESS
Total time: 9.167 s
```

Fuente: evidence/execution.log y evidence/junit-summary.txt. La duración de suite incluye preparación; Allure informa 2,261 s para el caso.

Ejecución en el IDE: requisito pendiente

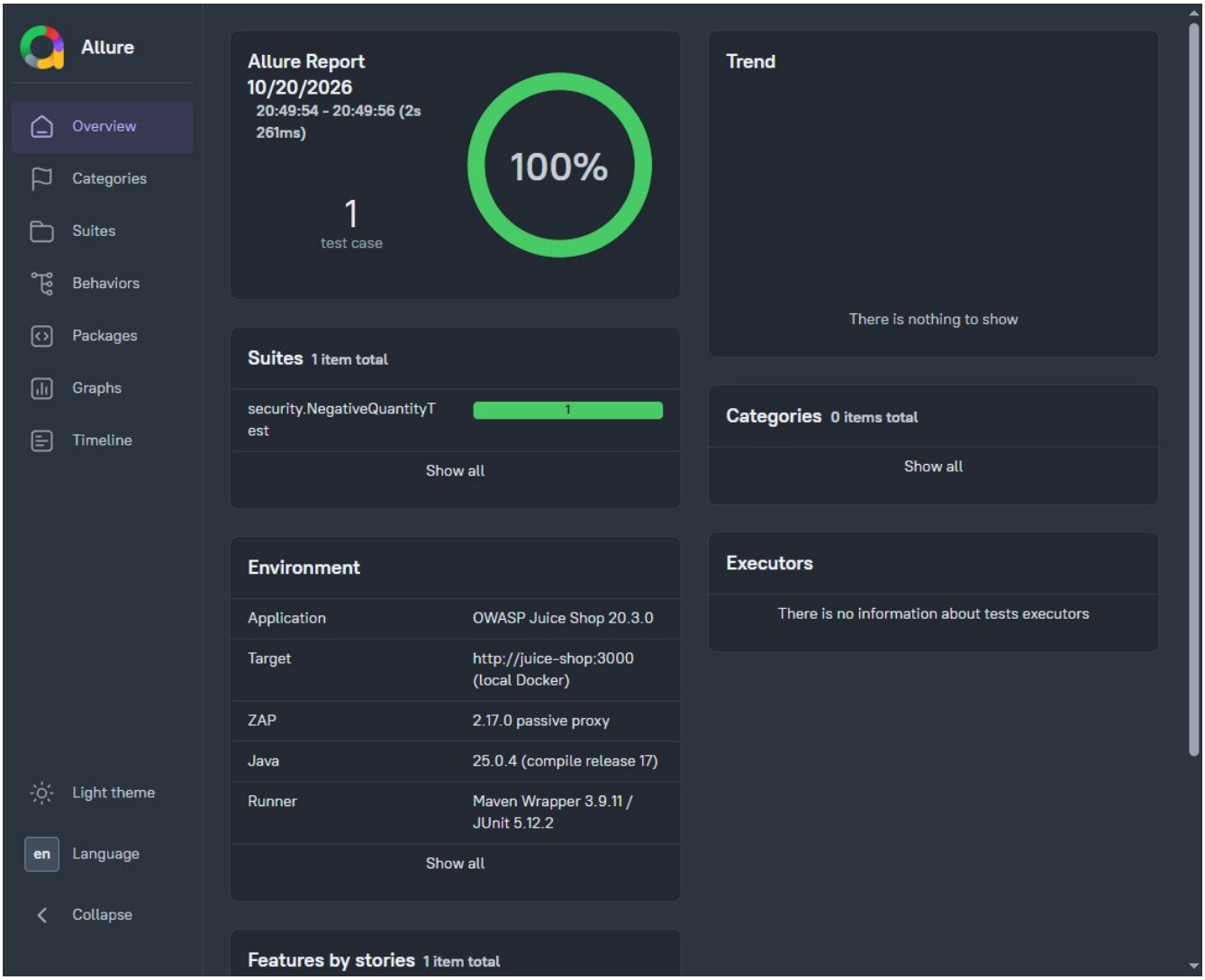
IntelliJ IDEA 2026.2.3 está instalado. La herramienta de escritorio falló al capturar su ventana en dos intentos (FrameArrived / window capture timed out). La ejecución por consola no se presenta como ejecución verificada dentro del IDE.

Para completar la evidencia: abrir pom.xml como proyecto Maven; seleccionar JDK 17+; iniciar los contenedores; ejecutar la configuración compartida TC-BL-001 - Juice Shop y capturar el resultado con 1 prueba aprobada.

INFORME TÉCNICO / 04

Reporte Allure: resumen

Reporte generado con Allure CLI 2.46.1 a partir de los resultados reales del adaptador allure-junit5 2.29.1. La captura corresponde a la ejecución final, sin resultados antiguos mezclados.

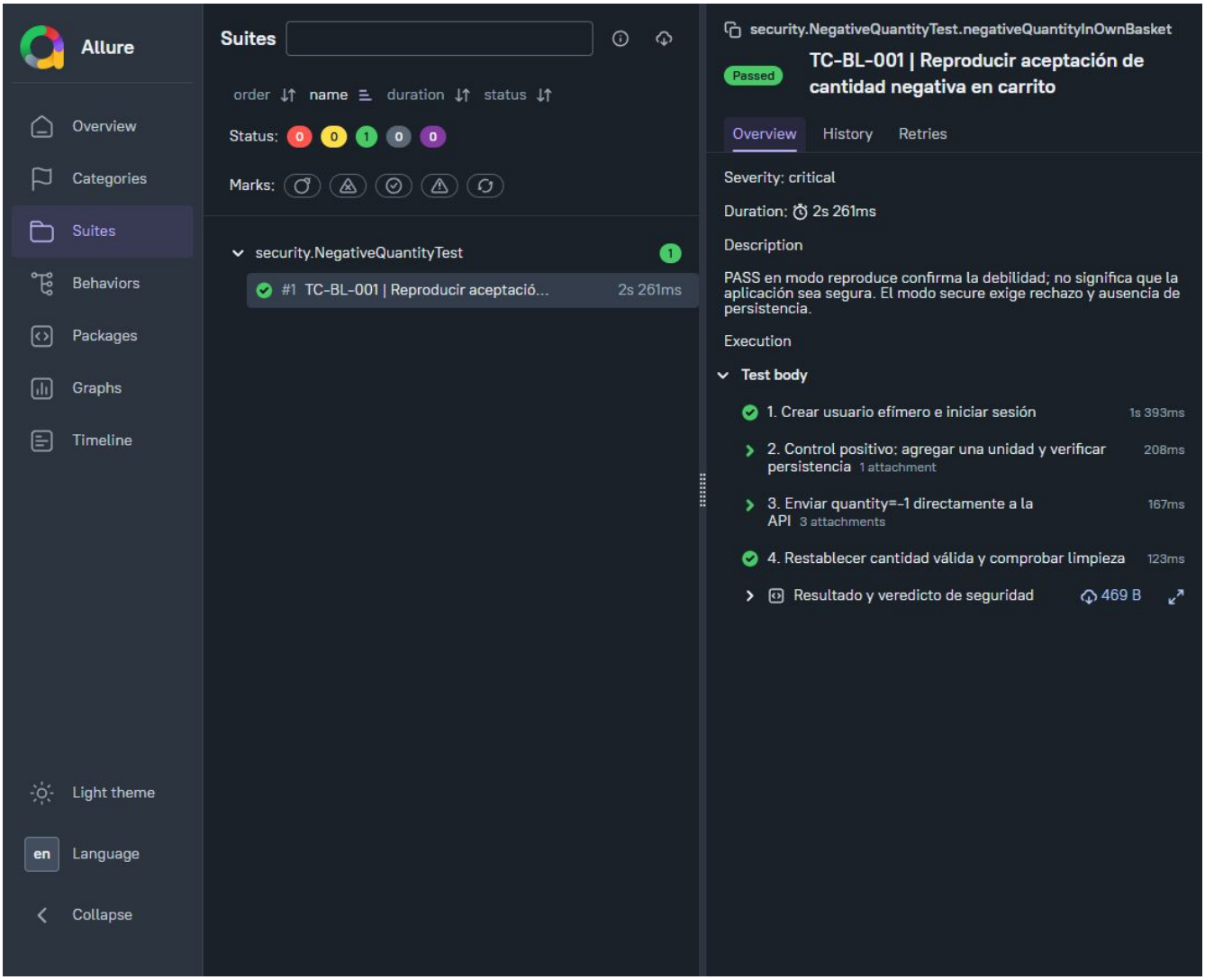


Métrica	Valor
Total / aprobadas	1 / 1
Fallidas / rotas / omitidas	0 / 0 / 0
Duración reportada por Allure	2.261 segundos
Interpretación	100% significa reproducción exitosa de la debilidad; no aprobación de seguridad.

Reporte interactivo incluido en output/allure-report/index.html. Abrir con allure open output/allure-report o mediante un servidor HTTP local; no requiere credenciales.

INFORME TÉCNICO / 05

Reporte Allure: pasos y adjuntos

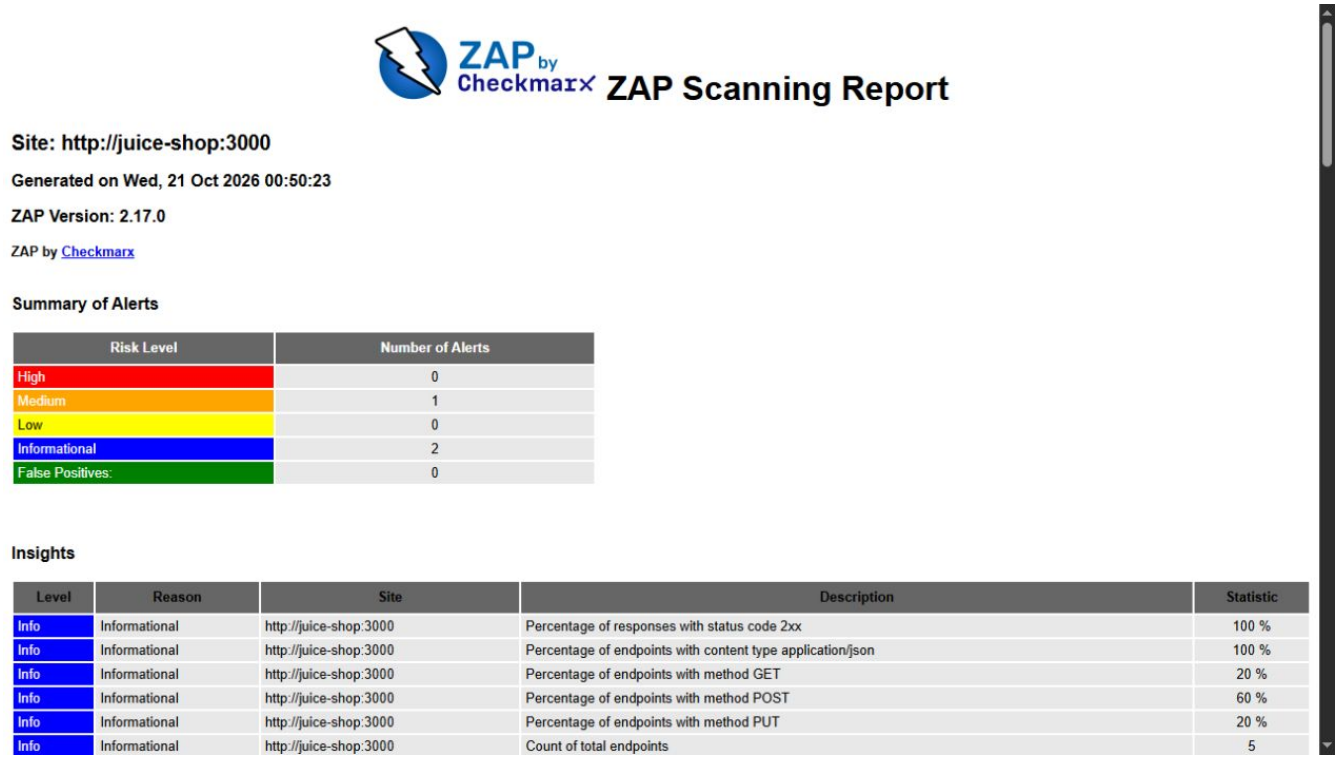


El caso conserva el control positivo, la petición manipulada, la respuesta y la lectura independiente del carrito. También registra la restauración a una unidad y el resumen JSON del resultado.

Adjunto	Propósito
Carrito antes de la manipulación	Acreditar cantidad inicial 1.
Petición de prueba / respuesta	Acreditar quantity=-1 y la aceptación por la API.
Lectura independiente del carrito	Confirmar que el valor fue persistido.
Resultado y veredicto de seguridad	Identificar escenario, tiempos, datos observados y limpieza.

No se adjuntan cuerpos de registro o login a Allure. La etiqueta critical representa prioridad del caso de prueba y no una puntuación CVSS ni una severidad demostrada sobre un sistema de producción.

Reporte ZAP: análisis pasivo



ZAP by Checkmarx ZAP Scanning Report

Site: http://juice-shop:3000

Generated on Wed, 21 Oct 2026 00:50:23

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	0
Informational	2
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://juice-shop:3000	Percentage of responses with status code 2xx	100 %
Info	Informational	http://juice-shop:3000	Percentage of endpoints with content type application/json	100 %
Info	Informational	http://juice-shop:3000	Percentage of endpoints with method GET	20 %
Info	Informational	http://juice-shop:3000	Percentage of endpoints with method POST	60 %
Info	Informational	http://juice-shop:3000	Percentage of endpoints with method PUT	20 %
Info	Informational	http://juice-shop:3000	Count of total endpoints	5

Regla / ID	Riesgo ZAP	Instancias
Cross-Domain Misconfiguration / 10098	Medio	5
Authentication Request Identified / 10111	Informativo	2
Session Management Response Identified / 10112	Informativo	1

Son **3 tipos de alertas y 8 instancias**, no ocho vulnerabilidades distintas. Cola pasiva al exportar: **0**. El reporte nativo se generó a las 00:50:23 UTC del 21/10, equivalentes a las 20:50:23 del 20/10 en Bolivia.

CORS: ZAP observó Access-Control-Allow-Origin: * en las respuestas. Esto requiere revisar qué datos son públicos y cómo se autentican las peticiones; por sí solo no demuestra lectura de información autenticada desde otro origen.

Las reglas 10111 y 10112 identifican mecanismos de autenticación y sesión: son informativas. La vulnerabilidad de cantidad negativa la comprueba el código Java; no se atribuye al escáner.

Alcance: cinco endpoints del flujo, dentro del laboratorio local. No es una evaluación exhaustiva de Juice Shop. Los reportes exportados redactan JWT y contraseñas efímeras. Archivos: output/zap/zap-report.html y zap-alerts.json.

Reproducción y recomendaciones

Entorno utilizado

Componente	Versión
Juice Shop / ZAP	20.3.0 / 2.17.0 (imágenes Docker oficiales)
Java / Maven Wrapper	JBR OpenJDK 25.0.4; compilación release 17 / Maven 3.9.11
JUnit / RestAssured / Allure adaptador	5.12.2 / 5.5.6 / 2.29.1
Allure CLI / sistema	2.46.1 / Windows + Docker Desktop

Comandos desde la raíz del proyecto

```
$env:JAVA_HOME = "C:\ruta\al\jdk"
./scripts/run-lab.ps1
allure generate target/allure-results --clean -o output/allure-report
allure open output/allure-report

# Regresión: exigir comportamiento seguro (fallará en la versión actual)
./mvnw.cmd test -Dexpectation=secure

# Detener el laboratorio
docker compose stop
```

El script espera disponibilidad, reinicia solo la sesión ZAP del laboratorio y ejecuta el único caso. Requiere Python 3 para exportar ZAP y Allure CLI para generar el HTML. El proxy se escucha únicamente en 127.0.0.1:8090. Juice Shop se publica en 127.0.0.1:3000.

Corrección propuesta

Validar en el servidor que quantity sea un entero entre 1 y un máximo comercial definido, tanto al crear como al actualizar ítems. Rechazar antes de escribir; preservar la cantidad anterior y devolver 400/422 con un mensaje controlado.

Aplicar una restricción consistente en persistencia y volver a validar cantidad, disponibilidad y precio al confirmar la compra. Añadir una regresión que exija rechazo de -1. Revisar CORS según los orígenes y datos realmente necesarios.

Impacto y límites

La integridad del carrito queda comprometida y puede propagarse a cálculos posteriores. La prioridad de corrección es alta, pero no se asigna un CVSS ni se afirma fraude consumado: el alcance no incluyó la compra. El modo secure existe como oráculo de regresión; no se presenta como una ejecución adicional realizada.

INFORME TÉCNICO / 08

Repositorio, entrega y trazabilidad

Destino del código:
https://github.com/nohpedro/vulneravity_test

Estado: publicación y visibilidad pública no verificadas. La entrega local está preparada para publicación; el informe no contiene un enlace inventado ni afirma que el código ya esté accesible públicamente.

Requisito	Estado / evidencia
Un caso de flujo de negocio	Completado: TC-BL-001 con control positivo y limpieza.
Código reproducible	Completado: src/, pom.xml, Maven Wrapper, compose.yaml y scripts/.
Ejecución satisfactoria	Completada por consola: BUILD SUCCESS, 1 prueba aprobada.
Ejecución satisfactoria en IDE	Pendiente de captura y validación visual en IntelliJ.
Allure	Completado: HTML, adjuntos y capturas en este documento.
ZAP	Completado: reporte nativo HTML, JSON y resumen en este documento.
Repositorio público	Pendiente de publicación/verificación de acceso sin sesión.

Integridad de los archivos de evidencia

evidence/business-result.json

```
529526565e0a4d124a68305a065b2ec8
b2245da631d8f7032e935d3449011d3d
```

evidence/execution.log

```
2085042d974a8aa434c6cdabe29c0a21
15f84f6a3f8dfbd1cd0474e4da6f6245
```

output/zap/zap-alerts.json

```
29bf00ea1ac8c97d9389c0b40f551137
5499ed13ff92ae4741bf97266abf5212
```

src/test/java/security/NegativeQuantityTest.java

```
b845eff0411905e97f64797736d7950a
3931217839f4325d55c0730ffc6e3b8b
```

Los dos renglones de cada hash SHA-256 forman una sola cadena, sin espacios. Los digests de las imágenes Docker se conservan en evidence/image-digests.txt.

Fuentes y material de referencia

El PDF Automation-Vulnerability-v6.pdf se utilizó como material didáctico sobre pruebas de vulnerabilidad y ZAP. Las instrucciones efectivas de la entrega provienen de la solicitud del usuario y de su selección de Juice Shop; el contenido adjunto no se trató como autorización para otras acciones.

El ZIP api_v6-main.zip aportó la referencia tecnológica Java/RestAssured. Su configuración apunta a todo.ly y contiene credenciales fijas; no se copiaron esas credenciales. Todo.ly y Todoist no son el mismo servicio. La solución creada usa Maven para facilitar la ejecución reproducible en este entorno.

Instalación oficial de OWASP Juice Shop

https://help.owasp-juice.shop/part1/running.html

API oficial de ZAP y escaneo pasivo

https://www.zaproxy.org/docs/api/

Alerta CORS 10098

https://www.zaproxy.org/docs/alerts/10098/

Allure y JUnit Jupiter

https://allurereport.org/docs/junit-jupiter/

Código oficial de Juice Shop

https://github.com/juice-shop/juice-shop

Interpretación de la evidencia

Los valores HTTP, cantidades, tiempos, alertas y capturas proceden de la ejecución del laboratorio. Las recomendaciones son propuestas de corrección. La demo pública, Todoist, el checkout y el impacto financiero no fueron probados; no se extrapolan a ellos los resultados.

Para cerrar la entrega académica: verificar el repositorio sin iniciar sesión, publicar el código y reportes, y sustituir la condición pendiente del IDE por una captura real de la configuración ejecutada satisfactoriamente. Después regenerar este PDF con scripts/build_pdf.py.